



3.1. Data Filtering

Previous Section:

 [2.2. Data Indexing and Slicing](#)

Contents:

[1. Introduction to Data Filtering](#)

[2. Filtering Based on Comparison Operators](#)

[3. Filtering Based on Logical Conditions](#)

[4. Filtering Based on Subset Membership](#)

[5. Filtering Based on String Conditions](#)

[Summary](#)

[References](#)

In the previous section, we learned how to access specific rows and columns through indexing and slicing. However, in real-world data analysis, we often need something more flexible—we want to **retrieve only the records that satisfy certain conditions**. This process is known as **data filtering**.

For example, you might want to:

- Find students with a GPA greater than 3.5.
- Retrieve only AI majors.
- Identify students who are at least 20 years old.
- Search for names that start with a particular letter.

Pandas makes these tasks simple by allowing us to create **Boolean conditions** that evaluate each row in a DataFrame. Rows that satisfy the condition (`True`) are returned, while rows that do not (`False`) are excluded.

Throughout this section, we will use the following synthetic student dataset.

```
import numpy as np
import pandas as pd

# Reproducible randomness
np.random.seed(42)

names = [
    "James", "Chris", "Adam", "Marian", "Peter",
    "Kevin", "Linda", "Emma", "Sophia", "Daniel",
    "Noah", "Olivia", "Lucas", "Grace", "Henry",
    "Liam", "Ava", "Mia", "Nathan", "Chloe"
]

majors = ["AI", "IT", "CSC", "SW"]

n = 20

df = pd.DataFrame({
    "Name": np.array(names),
    "Age": np.random.randint(18, 21, n),
    "Major": np.random.choice(majors, n),
    "GPA": np.round(np.random.uniform(2.0, 4.0, n), 2)
})

print(df.head(10))
```

1. Introduction to Data Filtering

Data filtering allows us to **retrieve only the rows in a DataFrame that satisfy one or more conditions**.

The basic syntax is `df[filtering_condition]`

The `filtering_condition` produces a **Boolean Series**, where each row is assigned either: `True` if the row satisfies the condition. `False` otherwise.

For example,

```
df["GPA"] >= 3.5
```

Might produce something like this:

Index	Name
0	False
1	False
2	True
3	False
4	False
5	False

When this Boolean Series is placed inside the square brackets,

```
df[df["GPA"] >= 3.5]
```

Pandas returns only the rows whose corresponding Boolean value is `True`. Suppose the output looks like this for a couple of first rows:

Index	Name	Age	Major	GPA
2	Henry	20	IT	3.80
18	Olivia	18	SW	3.70
25	Olivia	20	CSC	3.95
27	Adam	18	SW	3.78
29	Nathan	19	AI	3.59
39	Chloe	20	SW	3.88

Pandas supports several types of filtering conditions:

- **Comparison conditions** using operators such as `>`, `<`, `>=`, `<=`, `==`, and `!=`.
- **Logical conditions** using `&` (AND), `|` (OR), and `~` (NOT).
- **Subset-based conditions** using methods such as `isin()`, `any()`, and `all()`.
- **String conditions** using methods such as `str.contains()`, `str.startswith()`, and `str.endswith()`.



Note: Pandas also provides the `filter()` method.

- Despite its name, `filter()` **does not filter rows based on conditions**.
- Instead, it is primarily used to **select rows or columns by their labels** (names) using parameters such as `items`, `like`, or `regex`.

The Boolean indexing approach (`df[condition]`) is still the standard and most widely used method for filtering data.

The following sections introduce each of these filtering techniques.

2. Filtering Based on Comparison Operators

Comparison operators compare each value in a column against a specified value.

Operator	Meaning
<code>></code>	Greater than
<code><</code>	Less than
<code>>=</code>	Greater than or equal to
<code><=</code>	Less than or equal to
<code>==</code>	Equal to
<code>!=</code>	Not equal to

- **Example 1: Students with GPA greater than 3.5**

```
df[df["GPA"] > 3.5]
```

- **Example 2: Students aged 20**

```
df[df["Age"] == 20]
```

- **Example 3: Students who are not in the AI major**

```
df[df["Major"] != "AI"]
```

These comparisons evaluate every row independently and return only the matching records.

3. Filtering Based on Logical Conditions

Often, a single comparison is not enough. We may need to combine multiple conditions. Pandas supports three logical operators:

Operator	Meaning	Description
&	AND	Both conditions must be <code>True</code> .
	OR	Only one condition needs to be <code>True</code> .
~	NOT	The <code>~</code> operator reverses the Boolean values (<code>True</code> becomes <code>False</code> and vice versa).

Important: Each comparison must be enclosed in parentheses when using `&` or `|`.

- Return students who are in the AI major **and** have a GPA greater than 3.5.

```
df[(df["Major"] == "AI") & (df["GPA"] > 3.5)]
```

- Return students who are either AI majors **or** have a GPA above 3.8.

```
df[(df["Major"] == "AI") | (df["GPA"] > 3.8)]
```

- Return students who are **not** AI majors.

```
df[~(df["Major"] == "AI")]
```

4. Filtering Based on Subset Membership

Sometimes we want to check whether a value belongs to a collection of possible values instead of comparing against a single value.

Method	Description
<code>isin()</code>	Checks whether each value belongs to a specified list.
<code>any()</code>	Returns <code>True</code> if at least one Boolean value is <code>True</code> .
<code>all()</code>	Returns <code>True</code> only if every Boolean value is <code>True</code> .

Unlike `isin()`, the `any()` and `all()` methods summarize Boolean conditions into a single result rather than returning filtered rows.

- Return students majoring in AI or CSC.

```
df[df["Major"].isin(["AI", "CSC"])]
```

This is equivalent to `df[(df["Major"] == "AI") | (df["Major"] == "CSC")]` but is much cleaner when checking multiple values.

- Return True if at least one student has a GPA greater than 3.9.

```
(df["GPA"] > 3.9).any()
```

- If every generated student is at least 18 years old, the result is `True`.

```
(df["Age"] >= 18).all()
```

5. Filtering Based on String Conditions

Pandas provides the `.str` accessor for performing operations on text columns. Some commonly used string filtering methods include:

Method	Description
<code>str.contains()</code>	Checks whether a string contains a substring
<code>str.startswith()</code>	Checks whether a string begins with a prefix
<code>str.endswith()</code>	Checks whether a string ends with a suffix

- **Example 1: Names containing "an"**

```
df[df["Name"].str.contains("an")]
```

- **Example 2: Names starting with "A"**

```
df[df["Name"].str.startswith("A")]
```

- **Example 3: Names ending with "a"**

```
df[df["Name"].str.endswith("a")]
```

These methods are especially useful when working with names, addresses, emails, product descriptions, and other text-based data.

Summary

This section introduces to Data filtering and various methods to perform filtering on data.

- Data filtering retrieves only the rows that satisfy specified conditions.
- The syntax `df[condition]` works because the condition produces a Boolean Series of `True` and `False` values.
- Comparison operators (`>`, `<`, `>=`, `<=`, `==`, `!=`) filter data based on numeric or categorical values.
- Logical operators (`&`, `|`, `~`) combine multiple filtering conditions.
- `isin()` provides a convenient way to test whether values belong to a set of possible values.

- `any()` and `all()` summarize Boolean conditions into a single `True` or `False` result.
- String methods such as `str.contains()`, `str.startswith()`, and `str.endswith()` make it easy to filter text data.
- Together, these filtering techniques form one of the most important tools for exploring and analyzing datasets in Pandas.

To practice further, complete Lab 03 of the Data Analytics Hands-On Lab.

 [Lab 07: Data Analytics with Python](#)

References

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.filter.html>

<https://builtin.com/data-science/pandas-filter>

Next Section:

 [3.2. Sorting Data Values](#)